DeadlineLens — Project Proposal and Plan (PA#1)

Team: 23127001 — Nguyễn Lê Quan Anh (Member A) · 23127121 — Bùi Hữu Thịnh (Member T) · Repository: github.com/bhuuthinh2005-design/Deadline-Extractor · 22 Oct 2026

1. Product and users

User. Linh, a second-year FIT-HCMUS student taking four courses plus two group projects. Deadlines reach her as informal Vietnamese text in Zalo class groups, Moodle announcements and emails: “nộp bài trước 23h59 thứ 6 tuần sau”, “thầy dời deadline sang chiều thứ 4”. **Problem (one sentence):** deadlines and changes to them arrive as relative, chat-style Vietnamese, and Linh misreads or misses them. **Today** she retypes them into Google Calendar or Notion by hand, or scrolls back through Zalo; a later “dời deadline” message is the one most often lost. **Product:** a web app where she pastes a message and gets a calendar-ready deadline she confirms in one click.

2. The LLM feature and the cost of being wrong

Feature (one): deadline extraction. Input: pasted message + the time it was sent + the semester start date. The LLM returns JSON: course, task, event type (*new / change / cancel*), the due time as a *relative expression* (weekday, week offset, clock time), a verbatim source quote, and a status of *ok* or *needs_clarification*. Plain code (not the LLM) turns the expression into an absolute Asia/Ho_Chi_Minh datetime. The app shows “Thứ 6, 06/11/2026 23:59” beside the highlighted quote; nothing enters the calendar until Linh confirms.

Cost when wrong. *Who:* Linh, and her teammates when the calendar is shared. *How much:* a date that is too late, or a missed “dời sớm hơn” message, makes her submit after the real deadline, losing marks to the late penalty or the assignment altogether; a date that is too early only wastes effort. The errors are asymmetric, so we track **false-late** separately. *Undoable?* The calendar entry is editable; the missed deadline is not. *How we would know:* a hand-labelled gold set (target ≥150 messages: 100 real anonymised class messages, 50 synthetic hard cases) scored on exact-datetime match, false-late rate, wrong-confident rate, and abstain rate. Planned targets: false-late ≤2%, and wrong answers with status ok ≤3%; abstaining is preferred to guessing (“tuần sau” without a day, no time of day).

3. Scope for the semester

In scope	Deliberately out
Paste-in text box (Vietnamese); extraction of new / change / cancel deadlines; deterministic date resolver with unit tests; confirm-or-edit screen with quote highlighting; per-user list and calendar view; .ics export; change/cancel linking to an existing deadline; gold set + evaluation script run on every prompt change.	Reading Zalo/Moodle/email automatically (no scraping or integrations — paste only); two-way Google Calendar sync; reminders and push notifications; screenshots/OCR; English messages; shared group calendars; mobile app; fine-tuning a model; any grade or workload advice.

4. Plan across six checkpoints

#	Checkpoint	Date*	Owner	Work assigned
1	Proposal + plan (this)	Thu 29 Oct	A + T	A: scope, risks, tech; T: LLM section, cost-of-error. Start gold-set collection (30 messages each).
2	Specification (IA#1)	Thu 12 Nov	T (lead), A	T: extraction JSON contract, abstain rules, resolver cases; A: API and screen contract, acceptance criteria.
3	Core feature + gold set v1	Thu 26 Nov	T (pipeline), A (UI)	T: prompt + LLM call + date resolver; A: paste/confirm UI and database; both: finish ≥150 labelled messages, cross-check 30.
4	Evaluation (PA#4)	Thu 10 Dec	T (harness), A (analysis)	T: eval script, metrics, prompt iterations; A: error analysis by category, false-late report.
5	Feature-complete	Thu 24 Dec	A (calendar/.ics), T (change/cancel)	A: calendar view, .ics export, deploy; T: link change/cancel messages to earlier deadlines; regression eval.
6	Final demo + report	Thu 7 Jan	A (report), T (demo)	Final eval re-run, demo script, report on what changed and why; freeze scope on 3 Jan.

*Dates are our working dates, set from the semester so far; we will move them to the official course calendar and note any change.

5. The two risks most likely to sink this

R1 — No trustworthy gold set, so we cannot show the feature works. Real class messages are private and labelling is slow. *Start this week:* each of us exports 30 messages from our own class groups with names and phone numbers removed (and asks the group before using any), writes 25 synthetic hard cases (relative dates, changes, cancellations, ambiguous times), and we label in a shared sheet; both label the same 30 to measure agreement. Gate: at least 100 labelled by 26 Nov, otherwise we cut to course-title-only extraction.

R2 — The model computes the wrong date and looks confident. LLMs are unreliable at “thứ 6 tuần sau” arithmetic and academic-week numbering. *Mitigation:* the LLM only outputs a relative expression; a small resolver library does the calendar arithmetic. This week T writes 40 resolver unit tests (month ends, Sunday as week boundary, “tuần 8” against the semester start) and a rule that missing day or time returns *needs_clarification*. If the resolver passes but the model still mislabels, we narrow supported phrasings and say so in the app.

6. Technology choices

Choice	Why, for this project
TypeScript + Next.js (UI and API routes)	One language and one repo for two people; a paste-and-confirm UI and a handful of endpoints need nothing heavier.
Supabase (Postgres)	Stores deadlines and the labelled messages; hosted free tier avoids running our own database; SQL makes “change refers to deadline X” simple.
LLM: Google Gemini Flash-Lite class model via the Gemini API (schema-constrained JSON output)	Cheap enough to run the full gold set on every prompt change: a message is about 700 tokens, so 10,000 calls ≈ 7M tokens ≈ US$1–5 at list price (≈US$0.10–0.30 per 1M input; price to be re-checked at ai.google.dev), plus a free tier for development. Budget cap US$10 for the semester. Fallback: a second provider behind one interface if Vietnamese quality is poor.
date-fns / own resolver module (Asia/Ho_Chi_Minh)	Keeps date arithmetic deterministic and unit-testable (see R2).
Vitest + a CLI eval script; GitHub Actions	Runs unit tests and the gold-set metrics on every pull request so a prompt change that raises false-late is visible.
Vercel	Free deploy from the repository for the demo.